

JavaCUP 与 GNU Bison 差异比较

梁力航 23336128

2026 年 5 月 20 日

本文档对应 ROSE 项目实验三的提交要求：讨论 JavaCUP 和 GNU Bison 两种 yacc 族工具在语法规则定义方面的差异，并扼要指出 JavaCC 与 JavaCUP 的最核心区别。

1 JavaCUP 与 GNU Bison 语法规则定义差异

1.1 输入文件格式

	JavaCUP (.cup 文件)	GNU Bison (.y 文件)
终结符声明	<code>terminal SYM1, SYM2;</code>	<code>%token SYM1 SYM2</code>
非终结符声明	<code>non terminal NT;</code>	无需显式声明
开始符号	<code>start with symbol;</code>	<code>%start symbol</code>
优先级	<code>precedence left PLUS;</code>	<code>%left PLUS</code>
类型标注	<code>terminal String ID;</code>	<code>%type <str> ID</code> (配合 <code>%union</code>)
嵌入代码	<code>parser code { : ... : };</code>	<code>%{ ... %}</code>
语义动作语法	<code>{ : ... : }</code>	<code>{ ... }</code>

1.2 语义动作中符号属性的引用方式

这是两个工具最显著的差异。

JavaCUP 使用命名标签引用产生式右侧符号：

```
expression ::= expression:left PLUS expression:right
{ : RESULT = left + right; : }
```

GNU Bison 使用位置编号 \$1, \$2, ... 引用:

```
expression: expression PLUS expression
{ $$ = $1 + $3; }
```

JavaCUP 的命名标签在长规则中可读性更好——当产生式右侧有 8–10 个符号时, \$7 容易对应错误, 而 `identifier:id` 则一目了然。

1.3 类型系统

- **JavaCUP**: 每个非终结符和带类型的终结符单独声明类型
- **GNU Bison**: 通过 `%union` 定义所有可能的属性类型, 再用 `%type` 为各符号指定联合体字段名

1.4 生成文件

- **JavaCUP**: 生成 `Parser.java` (语法分析器) 和 `sym.java` (终结符常量)
- **GNU Bison**: 生成 `*.tab.c` (语法分析器) 和 `*.tab.h` (终结符常量, 可选)

1.5 平台与生态

- **JavaCUP**: 纯 Java 实现, 与 JFlex 通过 `%cup` 指令直接集成, Scanner 返回 `Symbol` 对象
- **GNU Bison**: C/C++ 实现, 与 GNU Flex 通过 `YYSTYPE` 联合体集成

2 JavaCC 与 JavaCUP 的最核心区别

JavaCC 生成的是递归下降 (自顶向下) 的 **LL(k)** 解析器, **JavaCUP** 生成的是移进/归约 (自底向上) 的 **LALR(1)** 解析器。

具体差异:

- **JavaCC**: 将 EBNF 语法规则直接转换为递归函数调用, 语义动作可嵌入在规则的任意位置。更接近手写递归下降的体验
- **JavaCUP**: 将语法规则转换为 LALR 分析表, 语义动作只能在产生式规约时执行。适合处理更大类的上下文无关文法

3 实际使用体会

1. 命名标签 (JavaCUP) 优于位置编号 (Bison), 这在 Oberon-0 较复杂的 `module` 规则中特别明显

2. Java 的类型系统在编译期提供类型安全保证——如果在 cup 文件中声明了 `terminal String ID;`，生成代码中类型不匹配会直接编译失败
3. JavaCUP 的冲突报告（shift/reduce, reduce/reduce）格式与经典 yacc 一致，可直接借鉴 yacc 的冲突解决经验
4. 与 JFlex 的集成是 Java 生态的优势——两个工具共享 `Symbol` 类型，无需手动桥接